

面向预测不确定性的微网购电与储能滚动调度

摘要

本文研究含光伏、储能与外部电网的微网购电调度问题。统一建立 10 分钟步长的能量平衡与储能状态线性规划模型，并核验 365 日、每日 144 时段的数据。问题一在确定性条件下得到购电费 35,126.95 元，较无储能基线降低 26.90%。问题二采用严格向前的同星期与近 7 日中位数融合预测，叠加历史净负荷残差 80% 分位裕量；将 1 月作为预测与 SOC 热身期后，2 月至 12 月 334 日费用为 15,386,279.80 元，紧急购电 427,890.01 kWh。问题三冻结已执行决策、继承 SOC，并穷举 6:00、12:00、18:00 三类更新时间的 8 种组合；计入正向调整费用后，不调整组合以 15,820,108.80 元取得最低总费用，说明新增预报并不必然产生净经济收益。问题四仅用历史价格预测未来电价，价格预测 MAE 为 0.0532 元/kWh；问题 4-2 费用为 16,108,523.19 元，问题 4-3 同样由不调整组合以 16,569,050.38 元最优。全部 6,570 项逐日与逐组合约束回代均通过。结果表明，严格信息集、跨日状态连续和调整成本显式化，是获得可实施调度结论的关键。

关键词：微网调度 线性规划 滚动预测 储能 鲁棒裕度

一、问题重述

某微网由小区负荷、光伏发电、储能装置及外部电网构成。外部电网负责补足本地供能缺口，储能可在低价或光伏富余时充电，在高价或供电不足时放电。题目给出固定日内电价与典型日曲线、全年负荷和光伏实测数据、每日四次光伏预报以及全年波动电价，要求制定经济且可执行的购电策略。

问题一要求在负荷、光伏和电价均已知时安排 144 个 10 分钟时段的购电与储能动作；问题二要求在每日 0:00 仅凭历史信息制定全天计划，并以 5 倍电价补充紧急购电；问题三允许在 6:00、12:00 和 18:00 依据新增预报调整尚未执行的购电量；问题四用波动电价分别重算问题二和问题三。各问共享储能物理约束，但信息集和结算方式逐层扩展。

本文总体技术路线见图 1。核心原则是先统一数据口径和物理可行域，再逐层增加预测误差、滚动更新与价格波动，最后以独立回代和简单基线检验模型收益。

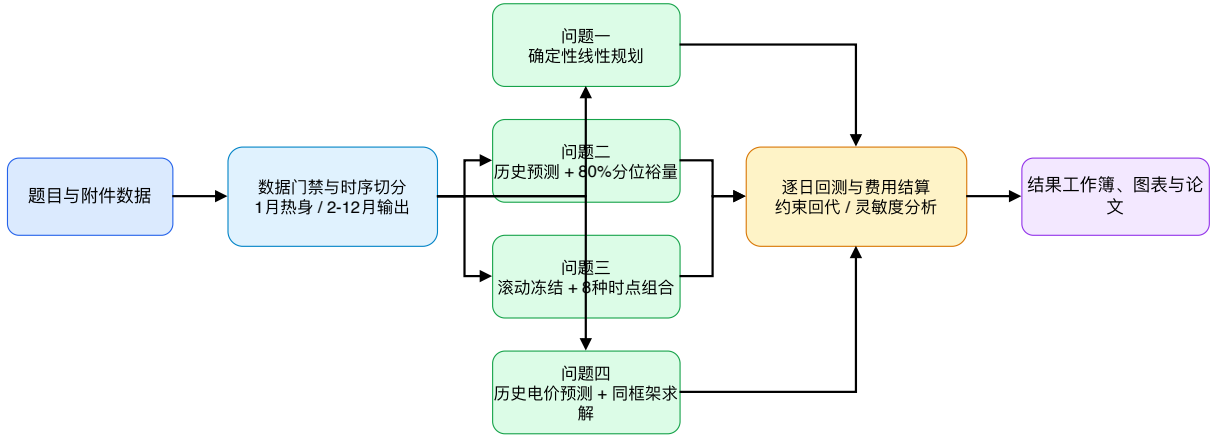


图 1: 微网购电与储能协同调度总体技术路线

二、数据处理与问题分析

2.1 数据门禁与统一口径

全年负荷、光伏和波动电价数据均为 365×144 ，每日四次光伏预报形成 1,460 个唯一“日期-发布时刻”键。检查结果为数值缺失 0、负值 0、重复日期 0。负荷范围为 1,995.7176–7,978.8849 kW，光伏为 0–10,216.2000 kW，波动电价为 0.0076–1.7936 元/kWh。夜间光伏零值具有物理意义，予以保留；光伏预报表的日期结构空白采用向下填充恢复。

功率进入能量平衡前按

$$\Delta t = \frac{1}{6} \text{ h}, \quad E_{d,t} = P_{d,t} \Delta t \quad (1)$$

转换为时段电量。原始 144 点按官方结果模板的行序一一对应，另以整体平移 ± 10 分钟检验标签歧义。决策日 d 的模型只能读取 d 以前的实测数据及决策时已经发布的预报，

事后实测轨迹仅用于结算，避免未来信息泄漏。数据流程见图 2。

数据门禁与无泄漏建模流程

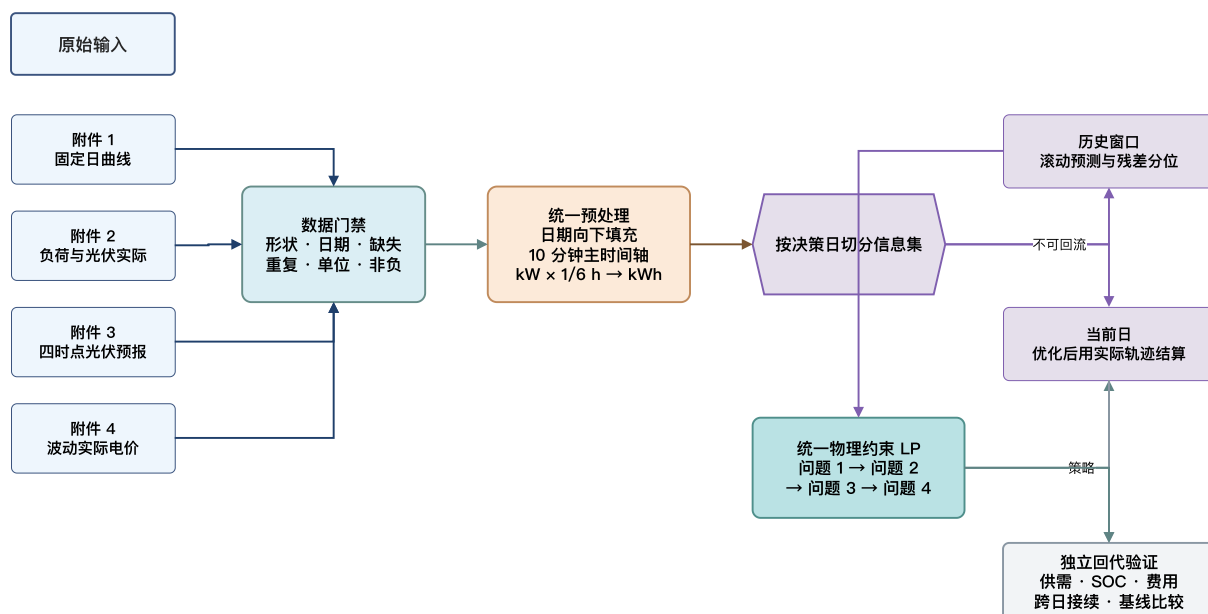


图 2: 数据门禁、信息集切分与无泄漏建模流程

2.2 四问递进关系

问题一提供确定性线性规划基座；问题二加入预测与紧急购电；问题三只对尚未执行时段滚动重算，并把“不调整”保留为可行基线；问题四只改变电价过程和其可见信息，不改变能量平衡。这样可将性能差异归因于新增信息或价格机制，而不是模型口径变化。

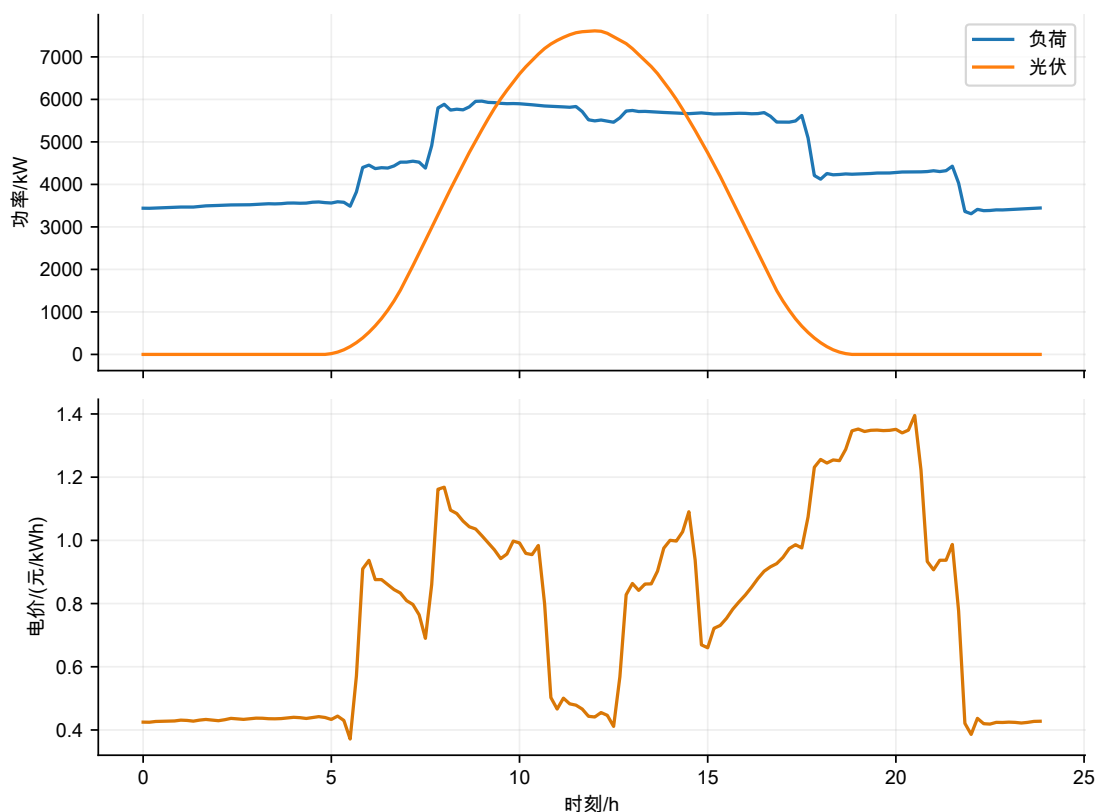


图 3: 附件一负荷与光伏功率的日内变化

三、 模型假设

1. 每个 10 分钟时段内负荷、光伏功率和电价保持常数，功率按式 (1)转换为电量。
2. 储能充电效率和放电效率均为 0.9，容量安全区间为 $[1200, 10800]$ kWh，交流侧充、放电功率上限均为 5,000 kW。
3. 微网不向外网售电；供给过剩允许弃置并显式记录。题目未给出并网购电功率上限，故不额外添加。
4. 问题一固定期初、期末 SOC 均为 6,000 kWh；问题二至问题四跨日继承 SOC，并要求每日优化终点不低于 6,000 kWh。
5. 决策时刻之后的实际负荷、光伏和波动电价不可见。所有预测由历史日顺序生成，不进行随机训练集划分。
6. 调整费用按“原计划全额，加相对 0:00 原计划的一次偏差费，再加紧急购电费”计算；已经执行的时段不可修改。

四、符号说明

表 1: 主要符号与单位

符号	含义	单位
d, t, r	日期、10 分钟时段、发布时刻	日, -, h
$L_{d,t}, G_{d,t}$	负荷、光伏功率	kW
$\pi_{d,t}$	电价	元/kWh
$p_{d,t}, g_{d,t}, e_{d,t}$	原计划、最终正常购电、紧急购电	kWh
$c_{d,t}, q_{d,t}$	充电量、放电量	kWh
$S_{d,t}$	时段末储能电量	kWh
$w_{d,t}$	弃光或过剩电量	kWh
η_c, η_d	充、放电效率, 均取 0.9	-
$\rho_{d,t}$	历史净负荷残差 80% 分位裕度	kWh

五、问题一：确定性购电与储能调度

5.1 统一物理模型

令净负荷电量为 $n_t = (L_t - G_t)\Delta t$ 。供需与 SOC 递推写为

$$g_t + q_t - c_t \geq n_t, \quad (2)$$

$$S_t = S_{t-1} + \eta_c c_t - q_t / \eta_d. \quad (3)$$

约束为 $1200 \leq S_t \leq 10800$, $0 \leq c_t, q_t \leq 5000/6$, 且 $S_0 = S_{144} = 6000$ 。代码以不等式 (2) 表示可弃置的富余供给, 等价于加入非负弃电变量后写等式。目标函数为

$$\min \sum_{t=1}^{144} \pi_t g_t + 10^{-7} \sum_{t=1}^{144} (c_t + q_t), \quad (4)$$

其中极小正则项只用于排除退化的同时充放电, 不改变两位小数费用结论。模型由 HiGHS 线性规划求解; 其底层高性能对偶修正单纯形方法见文献^[3]。求解后独立回代供需、SOC 和充放电互斥。

5.2 结果与基线比较

主模型购电费为 35,126.95 元, 全天购电 59,482.70 kWh; 无储能直接购电基线为 48,052.05 元, 节省 26.90%。储能充电 20,740.67 kWh、放电 16,799.94 kWh, SOC 由 6,000 kWh 回到 6,000 kWh, 且始终位于 1,200–10,800 kWh。图 4 表明, 储能在低价或光伏富余时段吸收电量, 并在高价时段释放。

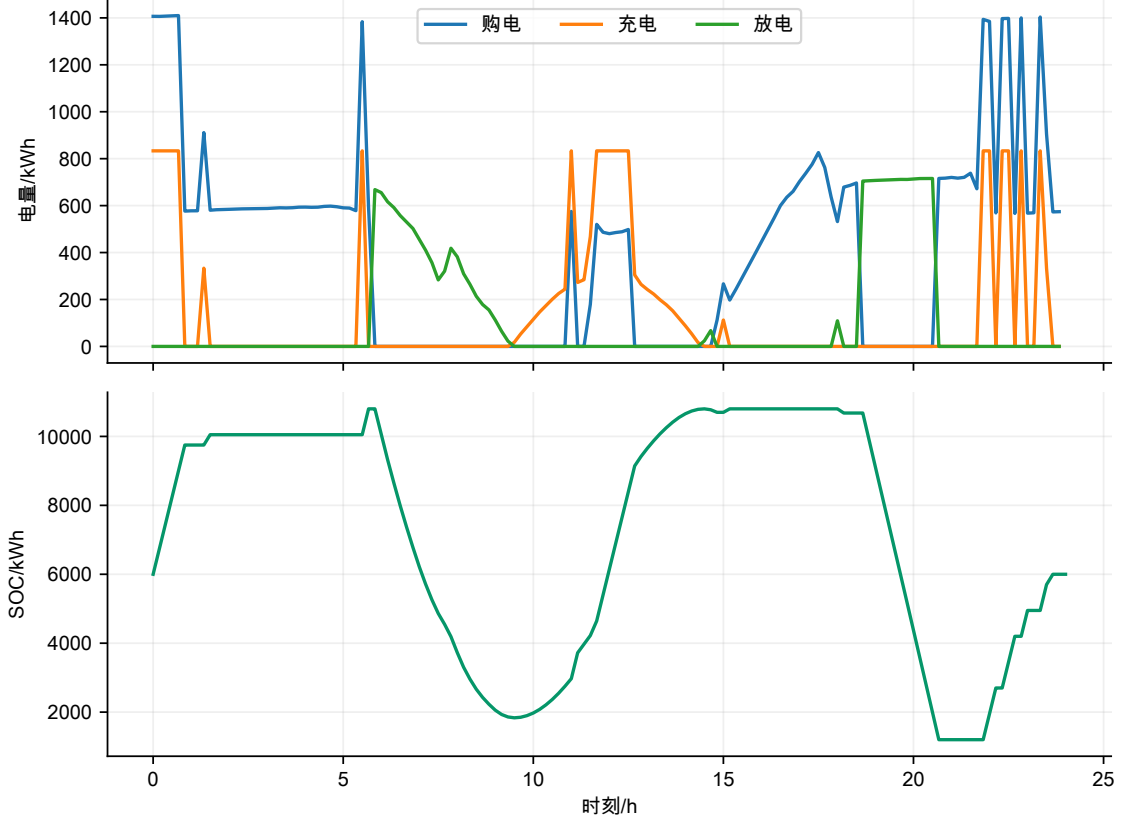


图 4: 问题一购电、充放电与 SOC 轨迹

六、 问题二：无泄漏预测与分位鲁棒调度

6.1 滚动预测

从 2025 年 1 月 1 日起连续推进 SOC，1 月作为预测与状态热身期，正式统计从 2 月 1 日开始。对变量 $x \in \{L, G, \pi\}$ ，同一时段的点预测为

$$\hat{x}_{d,t} = 0.65 \text{ med}\{x_{j,t} : j < d, j \equiv d \pmod{7}\} + 0.35 \text{ med}\{x_{j,t} : d - 7 \leq j < d\}. \quad (5)$$

同星期部分最多取最近 4 个同星期样本。负荷预测 MAE 和 WAPE 分别为 324.10 kW 和 7.01%；光伏分别为 203.79 kW 和 8.82%。图 5 展示一日预测实例。

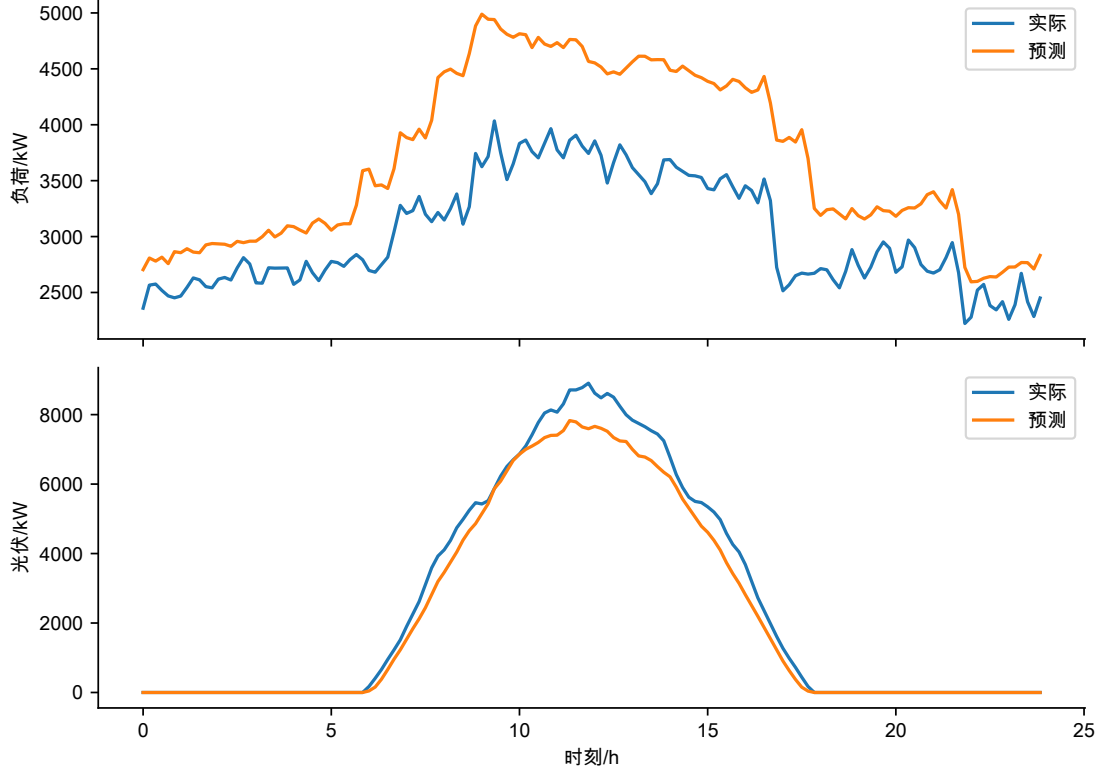


图 5: 负荷与光伏历史滚动预测示例

6.2 安全裕度与优化

将历史 28 日内有效预测残差 $u_{j,t} = n_{j,t} - \hat{n}_{j,t}$ 的 80% 分位记为 $\rho_{d,t}$, 以

$$\tilde{n}_{d,t} = \hat{n}_{d,t} + \rho_{d,t} \quad (6)$$

代入问题一的线性规划。实际供给不足部分

$$e_{d,t} = \max\{n_{d,t} - (g_{d,t} + q_{d,t} - c_{d,t}), 0\} \quad (7)$$

按 $5\pi_{d,t}$ 结算。该模型是场景风险模型的线性分位近似, 代码并未计算 CVaR 尾部值, 故本文不将结果表述为完整 CVaR 优化。

6.3 结果

334 日累计费用为 15,386,279.80 元, 紧急购电为 427,890.01 kWh。与使用事后真实负荷和光伏的完美信息下界相比, 本结果强调可实施性: 任一决策时刻均不读取未来实测值, 且日末 SOC 直接传递至下一日。

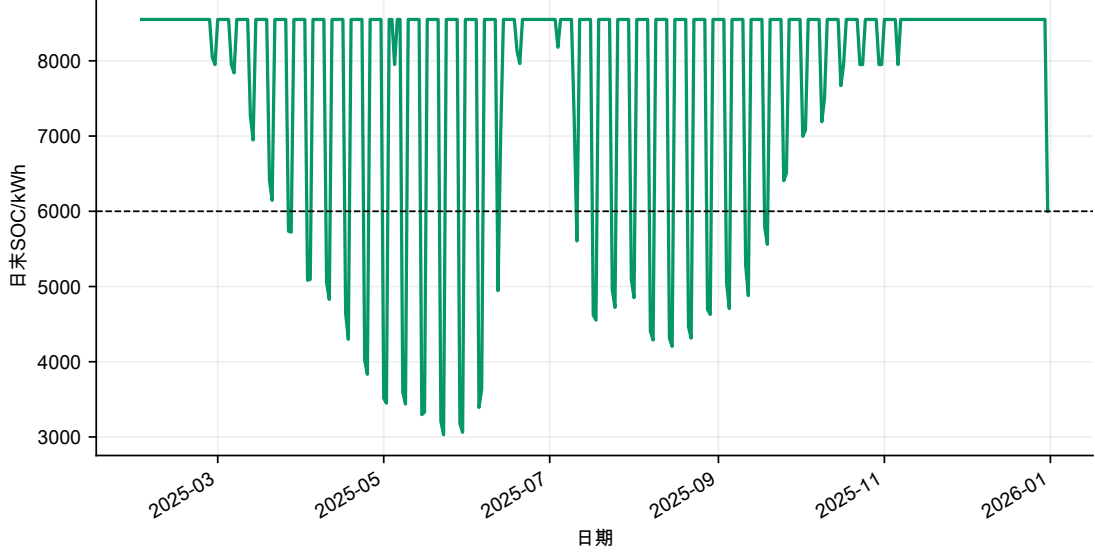


图 6: 正式统计期内储能日初与日末 SOC 连续轨迹

图 6显示 SOC 并未在月初或每日被人为重置, 这保证了 334 日费用来自一条连续可执行的状态路径。

七、 问题三：多时点滚动调整

7.1 滚动机制与费用

0:00 产生原计划 $p_{d,t}$, 6:00、12:00、18:00 将小时光伏预报线性插值到 10 分钟, 并用当日已观测负荷偏差修正剩余时段负荷。每次只优化未执行区间, 冻结历史决策并继承上一时段 SOC。这一做法属于微网能量管理中常见的滚动时域控制框架 [1, 2], 具体机制如图 7。

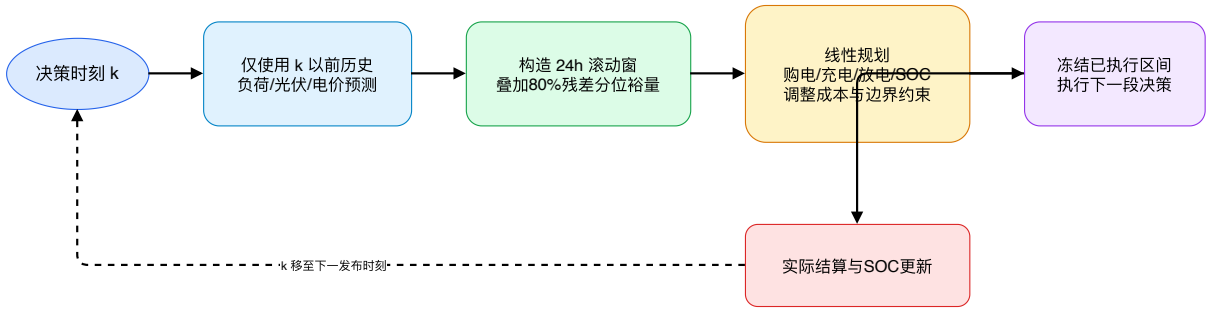


图 7: 多时点光伏预报驱动的滚动购电与储能调度机制

最终正常购电量为 $g_{d,t}$ 。按采用的结算解释, 每日总费用为

$$C_d = \sum_t [\pi_t p_t + 1.5\pi_t (g_t - p_t)^+ + 0.5\pi_t (p_t - g_t)^+ + 5\pi_t e_t]. \quad (8)$$

调整阶段的 LP 以相对原计划的上调量、下调量作为辅助变量，并保留问题一的 SOC 和功率约束。

7.2 更新时点组合选择

对集合 {6, 12, 18} 的全部 8 个子集逐一进行 334 日回测。累计费用由低到高的前三种组合为：不调整 15,820,108.80 元、仅 6:00 调整 16,139,224.08 元、仅 12:00 调整 16,496,525.79 元。因此正式结果选择“不调整”；这不是省略问题三，而是把是否调整纳入经济性筛选后得到的最优离散策略。最佳非空组合为仅 6:00 更新，其相对不调整的逐日费用差异见图 8。

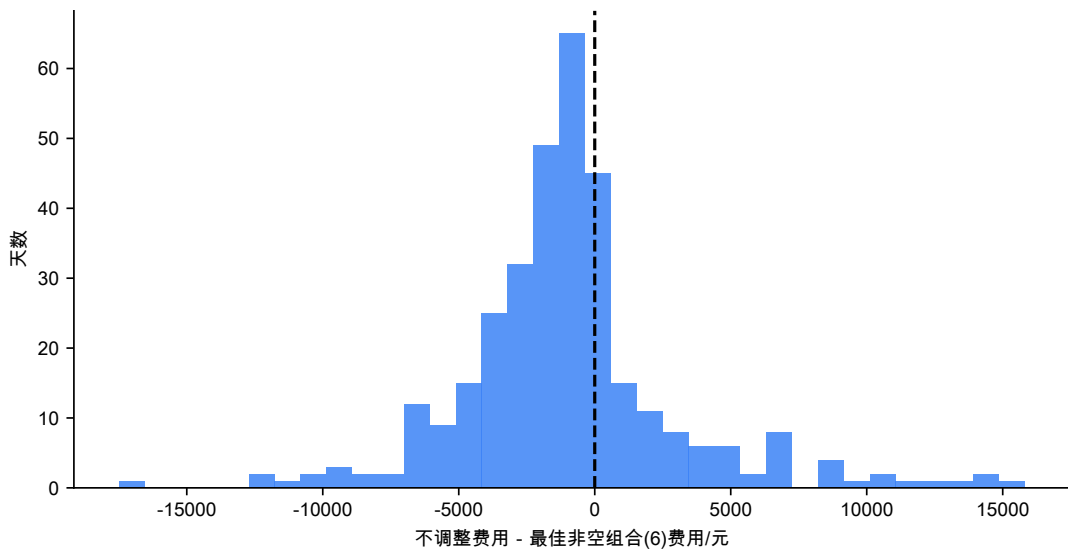


图 8: 问题三不调整与最佳非空更新组合的逐日费用差值

八、 问题四：波动电价下的扩展

8.1 价格预测与问题 4-2

未来实际价格不可见，故对电价同样采用式 (5) 的历史滚动预测，实际价格只用于事后结算。价格预测 MAE 和 WAPE 分别为 0.0532 元/kWh 和 6.94%，预测与实际日均价格见图 9。

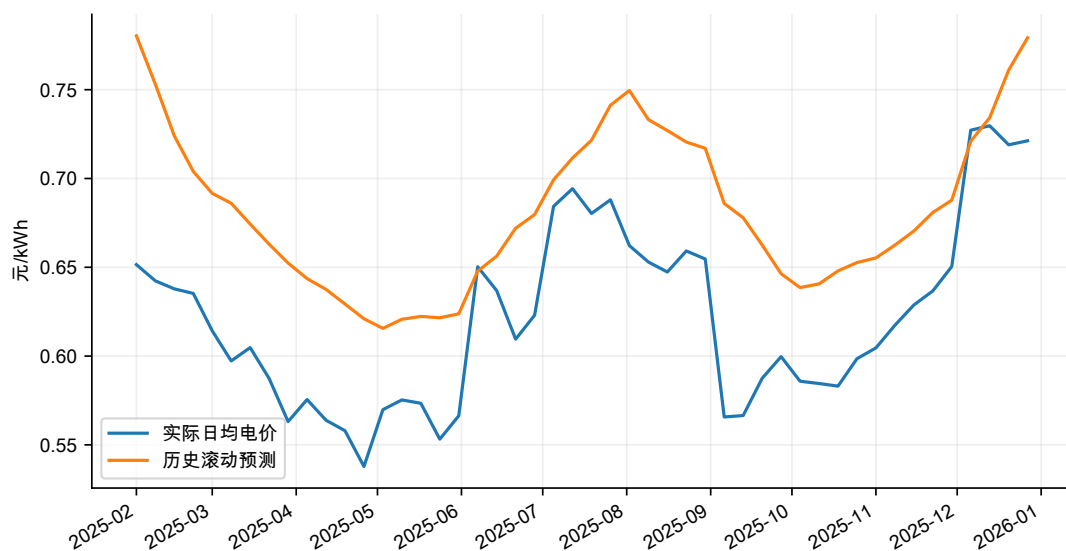


图 9: 波动电价的历史滚动预测与实际值

问题 4-2 将预测价格代入问题二的目标函数，按实际价格事后结算的累计费用为 16,108,523.19 元。该口径把预测价格用于决策、真实价格仅用于评价，避免把事后价格曲线泄漏进日前计划。

8.2 问题 4-3 与综合比较

问题 4-3 同时在发布时点更新光伏预测，并根据已观察价格偏差修正剩余时段价格预测。8 种组合中不调整以 16,569,050.38 元最低；最佳非空组合为仅 6:00 更新，费用 17,138,646.42 元。图 10 同时给出固定价与波动价下的组合排序，表明高调整罚金下全时点更新并非必然最优。

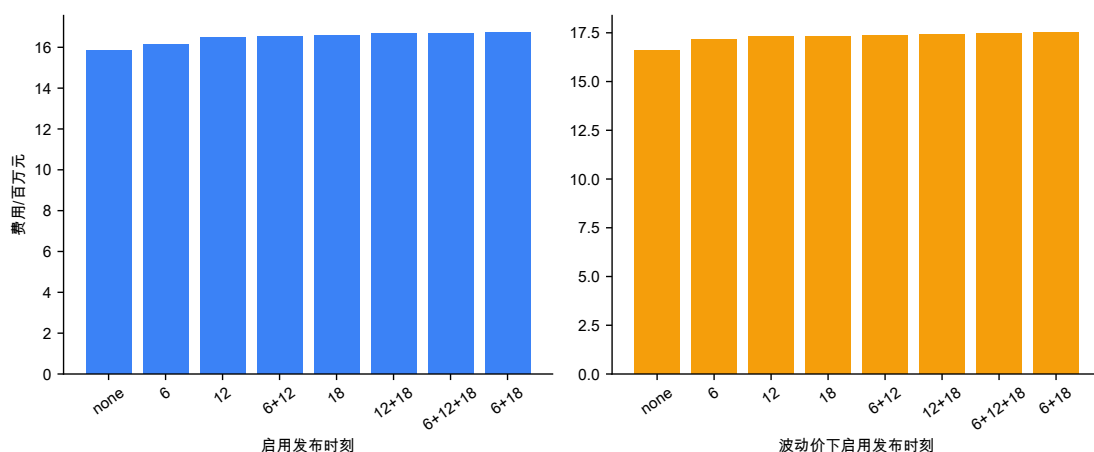


图 10: 固定价与波动价下 8 种更新组合累计费用

表 2: 四个指定日期的主模型总费用（元）

日期	问题二	问题三	问题 4-2	问题 4-3
2025-03-20	44,643.13	58,465.72	45,189.83	59,961.07
2025-06-21	31,552.20	30,583.44	29,674.44	26,238.07
2025-09-23	56,659.21	50,441.33	58,452.46	52,794.43
2025-12-21	66,252.63	66,821.64	77,718.02	78,830.69

九、 敏感性、稳健性与一致性检验

9.1 时间标签敏感性

将问题一的 144 点净负荷与电价整体平移 -10 、 0 、 $+10$ 分钟并重新优化，费用分别为 35,129.73、35,126.95 和 35,126.85 元；相对主口径最大偏差仅 2.78 元，说明总费用对这一行序歧义不敏感。指定时段的解释仍应遵循官方模板行序。

9.2 可行性与数值残差

求解前以零储能、直接购电构造可行基线；求解后独立回代供需、SOC、容量边界及费用分量。问题二、问题 4-2 及两类价格下 8 种更新组合共 6,570 项检查全部通过，通过率 100%；所有 SOC 均位于 $[1200, 10800]$ kWh，误差处于浮点求解容差内。

9.3 结果稳健性边界

问题一将充放电效率从 0.80 提高到 1.00 后重新求解，费用曲线见相应记录，验证效率改善会单调扩大套利空间。问题三与问题 4-3 已穷举 0/6/12/18 构成的 8 种发布组合，均由不调整取得最低费用，结论并非仅来自两个端点。完整场景 CVaR 和不同调整结算解释仍应作为后续扩展。

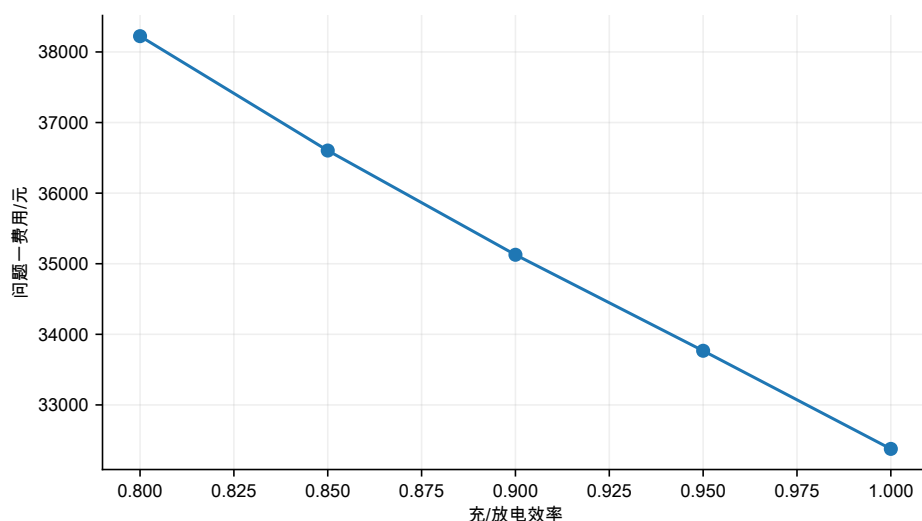


图 11: 问题一购电费用对储能充放电效率的敏感性

十、模型评价与结论

10.1 模型优点

本文以同一能量平衡和 SOC 递推贯穿四问，新增机制只改变信息集和费用项，便于归因比较。预测按日期严格前推，避免随机划分造成时间泄漏；从 1 月 1 日开始连续推进 SOC，避免 2 月 1 日人为重置状态；优化后独立回代 6,570 项约束并保存迭代记录。分位裕度将紧急购电代价转化为可解释的保守净负荷。

10.2 模型局限

分位鲁棒 LP 是完整场景风险模型的近似，未显式描述时序相关场景或输出 CVaR。储能互斥依赖正电价、效率损耗和极小吞吐正则项，而非二进制变量；在允许负电价或售电时需改为混合整数模型。组合比较属于给定样本内回测，若用于长期运营，还应进行跨年度外推验证。

10.3 结论与推广

确定性条件下，储能低充高放使典型日费用下降 26.90%；严格历史预测与 80% 残差分位裕度下，问题二 334 日费用为 15,386,279.80 元。固定价和波动价两种情形中，8 种组合均由“不调整”获得最低累计费用。因此实际运营应把“是否调整”作为带成本的离散决策，而不是收到新预报后机械重算。该框架可推广至园区综合能源、充电站和含可再生能源的企业微网。

十一、 人工智能工具使用声明

本队在竞赛期间使用 OpenAI Codex 辅助进行两版方法比较、程序调试、图表生成、文字与 LaTeX 排版、结果一致性验证及版本管理。模型选择、结算口径和最终结论由参赛队复核确认。详细提示范围、处理过程、采纳方式和验证记录见支撑材料中的《人工智能工具使用详情》。

十二、 参考文献

参考文献

- [1] Prodan I, Zio E. A model predictive control framework for reliable microgrid energy management[J]. International Journal of Electrical Power & Energy Systems, 2014, 61: 399–409. DOI: 10.1016/j.ijepes.2014.03.017.
- [2] Hu J, Shan Y, Guerrero J M, et al. Model predictive control of microgrids—An overview[J]. Renewable and Sustainable Energy Reviews, 2021, 136: 110422. DOI: 10.1016/j.rser.2020.110422.
- [3] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119–142. DOI: 10.1007/s12532-017-0130-5.

附录：核心实现与复现

支撑材料文件列表

支撑材料压缩包包含以下内容：

- `code/run_fusion.py`（融合流程入口）、`code/run_hybrid.py`（线性规划内核）与`code/README.md`；
- `code/outputs/records/`（参数、预测、逐日指标和约束检查）与`code/outputs/results/`（五个结果工作簿）；
- `figures/`（论文图及其 CSV 数据源）和《人工智能工具使用详情》PDF。

赛题给出的原始附件不重复收入支撑材料。

复现说明与核心接口

本文结果由同一程序从原始工作簿生成。随机种子固定为 20260910，核心参数为 $\Delta t = 1/6$ 小时、 $\eta_c = \eta_d = 0.9$ 、 $S \in [1200, 10800]$ kWh、充放电功率上限 5,000 kW、滚动残差分位数 0.80。核心 LP 接口如下：

```
def optimize_schedule(net_kwh, price, s0,
                      terminal_eq=None, terminal_min=6000,
                      plan_ref=None):
    # Variables: grid, charge, discharge, SOC.
    # Adjustment mode adds upward/downward deviations.
    res = linprog(cobj, A_ub=Aub, b_ub=bub,
                  A_eq=Aeq, b_eq=beq, bounds=bounds,
                  method="highs",
                  options={"dual_feasibility_tolerance": 1e-8,
                          "primal_feasibility_tolerance": 1e-8})
    if not res.success:
        raise RuntimeError(res.message)
    return checked_schedule(res.x)
```

在项目根目录执行以下命令可从零读取附件、完成数据门禁、求解四问、回代约束并重建结果工作簿与数据图：

```
MPLCONFIGDIR=.mplconfig python3 code/run_fusion.py
```

在 `paper` 目录连续执行两次以下命令可重建论文：

```
xelatex -interaction=nonstopmode main.tex
xelatex -interaction=nonstopmode main.tex
```

完整源程序

以下为生成全部结果、记录与数据图的完整可运行源程序。

```

#!/usr/bin/env python3
"""融合重算版：严格时序预测、跨日 SOC、发布时刻组合与波动价格。"""
from __future__ import annotations

import itertools
import json
import platform
import sys
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

import run_hybrid as core

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "code" / "outputs"
FIG = ROOT / "figures"
REPORT = ROOT / "reports" / "RESULTS_REPORT.md"
T = core.T
DT = core.DT

def forecast_asof(x: np.ndarray, target: int, cutoff: int) -> np.ndarray:
    """Forecast target-day profile using rows strictly before cutoff."""
    if cutoff <= 0:
        return np.zeros(T)
    recent = np.median(x[max(0, cutoff - 7):cutoff], axis=0)
    idx = np.array([j for j in range(max(0, cutoff - 35), cutoff) if j % 7 == target % 7], dtype=int)
    same = np.median(x[idx[-4:]], axis=0) if len(idx) else recent
    return 0.65 * same + 0.35 * recent

def prediction_matrices(x: np.ndarray) -> np.ndarray:
    return np.vstack([forecast_asof(x, i, i) for i in range(len(x))])

def margin_asof(actual_net: np.ndarray, pred_net: np.ndarray, cutoff: int, q: float = 0.8) -> np.ndarray:
    lo = max(1, cutoff - 28)
    if cutoff - lo < 7:
        return np.zeros(T)
    residual = actual_net[lo:cutoff] - pred_net[lo:cutoff]
    residual = residual[np.isfinite(residual).all(axis=1)]
    return np.quantile(residual, q, axis=0) if len(residual) else np.zeros(T)

def profile48(x: np.ndarray, i: int, pred: np.ndarray) -> np.ndarray:
    nxt = forecast_asof(x, i + 1, i) if i + 1 < len(x) else pred[i]
    return np.r_[pred[i], nxt]

def slice_schedule(s: core.Schedule, n: int = T) -> core.Schedule:
    g = s.g[:n].copy(); c = s.charge[:n].copy(); d = s.discharge[:n].copy(); soc = s.soc[:n + 1].copy()
    return core.Schedule(g, c, d, soc, s.objective, s.status, s.nit, s.runtime,
        s.balance_slack_min, s.soc_residual_max, s.simultaneous_max)

def zero_schedule(s0: float) -> core.Schedule:
    z = np.zeros(T); soc = np.full(T + 1, s0)
    return core.Schedule(z.copy(), z.copy(), z.copy(), soc, 0.0, 0, 0, 0.0, 0.0, 0.0, 0.0)

def simulate_q2(load, pv, price_actual, price_pred, load_pred, pv_pred, start_output=core.TEST_START):
    net_actual = (load - pv) * DT
    pred_net = (load_pred - pv_pred) * DT
    s0 = core.SOC_SAFE
    rows = []; metrics = []; checks = []; iterations = []
    for i in range(len(load)):
        margin = margin_asof(net_actual, pred_net, i)
        if i == 0:
            day = zero_schedule(s0)
        else:
            target = np.r_[pred_net[i] + margin,
                (forecast_asof(load, i + 1, i) - forecast_asof(pv, i + 1, i)) * DT + margin] if i + 1 < len(load) else
                pred_net[i] + margin

```

```

        pnext = forecast_asof(price_actual, i + 1, i) if i + 1 < len(load) else price_pred[i]
        ph = np.r_[price_pred[i], pnext] if i + 1 < len(load) else price_pred[i]
        full = core.optimize_schedule(target, np.maximum(ph, 1e-6), s0, terminal_min=core.SOC_SAFE)
        day = slice_schedule(full)
        settle = core.actual_settlement(day, net_actual[i], price_actual[i])
        s0 = day.soc[-1]
        checks.append(check_schedule(day, net_actual[i], settle["emergency"], f"Q2@{i}"))
        iterations.append([i, day.nit, day.runtime])
        if i >= start_output:
            date = str(pd.Timestamp(core_dates.iloc[i]).date())
            rows.append({"date": date, "plan": day.g, "charge": day.charge, "discharge": day.discharge,
                        "soc": day.soc, "emergency": settle["emergency"], "total_cost": settle["total_cost"]})
            metrics.append([date, settle["total_cost"], settle["plan_cost"], settle["emergency_cost"],
                           settle["emergency_kwh"], settle["waste_kwh"], day.soc[0], day.soc[-1]])
        return rows, pd.DataFrame(metrics, columns=["date", "total_cost", "plan_cost", "emergency_cost", "emergency_kwh", "waste_kwh",
            start_soc", "end_soc"]), checks, iterations

def pv_forecast_24(forecast, date: str, issue: int, current_kw: float) -> np.ndarray:
    vals = forecast[(date, f"{issue:00}")]
    return np.interp(np.arange(1, T + 1), np.arange(0, T + 1, 6), np.r_[current_kw, vals])

def horizon_profile(x, pred, i, issue, bias=0.0):
    cur = pred[i].copy()
    nxt = forecast_asof(x, i + 1, i) if i + 1 < len(x) else cur.copy()
    cur = np.maximum(cur + bias, 0)
    nxt = np.maximum(nxt + bias, 0)
    st = issue * 6
    return np.r_[cur[st:], nxt[st:]]

def q3_target(i, issue, load, pv, forecast, load_pred, pred_net, net_actual):
    date = str(pd.Timestamp(core_dates.iloc[i]).date())
    st = issue * 6
    lbias = float(np.median(load[i, :st] - load_pred[i, :st])) if st else 0.0
    lh = horizon_profile(load, load_pred, i, issue, lbias)
    current_pv = float(pv[i, st - 1]) if st else 0.0
    pvh = pv_forecast_24(forecast, date, issue, current_pv)
    margin = margin_asof(net_actual, pred_net, i)
    mh = np.r_[margin[st:], margin[st:]]
    return (lh - pvh) * DT + mh

def price_horizon(i, issue, price_actual, price_pred, update):
    st = issue * 6
    bias = float(np.median(price_actual[i, :st] - price_pred[i, :st])) if update and st else 0.0
    return np.maximum(horizon_profile(price_actual, price_pred, i, issue, bias), 1e-6)

def check_schedule(s, net_actual, emergency, tag):
    bal = s.g + s.discharge - s.charge + emergency - net_actual
    soc_calc = s.soc[:-1] + core.ETA_C * s.charge - s.discharge / core.ETA_D
    return {"scope": tag, "balance_min": float(bal.min()), "soc_residual_max": float(np.max(np.abs(s.soc[1:] - soc_calc))),
            "soc_min": float(s.soc.min()), "soc_max": float(s.soc.max()),
            "simultaneous_max": float(np.minimum(s.charge, s.discharge).max()),
            "passed": bool(bal.min() >= -1e-7 and np.max(np.abs(s.soc[1:] - soc_calc)) <= 1e-7 and s.soc.min() >= core.SOC_MIN -
                           1e-7 and s.soc.max() <= core.SOC_MAX + 1e-7)}

def simulate_q3_combo(combo, load, pv, forecast, price_actual, price_pred, load_pred, pv_pred, update_price=False):
    net_actual = (load - pv) * DT
    pred_net = (load_pred - pv_pred) * DT
    s0 = core.SOC_SAFE
    rows=[]; metrics=[]; checks=[]; iterations=[]
    issues = sorted(combo)
    for i in range(len(load)):
        date = str(pd.Timestamp(core_dates.iloc[i]).date())
        target0 = q3_target(i, 0, load, pv, forecast, load_pred, pred_net, net_actual)
        p0 = price_horizon(i, 0, price_actual, price_pred, update_price)
        base = core.optimize_schedule(target0, p0, s0, terminal_min=core.SOC_SAFE)
        plan = base.g.copy()
        fg=np.zeros(T); fc=np.zeros(T); fd=np.zeros(T); fs=np.zeros(T+1); fs[0]=s0
        starts=[0] + [h*6 for h in issues]; ends=[h*6 for h in issues] + [T]
        current=base
        for st,en in zip(starts,ends):

```

```

        if st:
            issue=st//6
            target=q3_target(i, issue, load, pv, forecast, load_pred, pred_net, net_actual)
            ph=price_horizon(i, issue, price_actual, price_pred, update_price)
            pref=np.r_[plan[st:], np.full(st, plan[-1])]
            current=core.optimize_schedule(target, ph, fs[st], terminal_min=core.SOC_SAFE, plan_ref=pref)
            n=en-st
            fg[st:en]=current.g[:n]; fc[st:en]=current.charge[:n]; fd[st:en]=current.discharge[:n]
            for t in range(st,en): fs[t+1]=fs[t]+core.ETA_C*fc[t]-fd[t]/core.ETA_D
            iterations.append([date, "+".join(map(str,issues)) or "none", st//6, current.nit, current.runtime])
            sched=core.Schedule(fg,fc,fd,fs,0,0,0,0,0,0,float(np.minimum(fc,fd).max()))
            settle=core.actual_settlement(sched,net_actual[i],price_actual[i],plan=plan,adjusted=bool(issues))
            s0=sched.soc[-1]
            checks.append(check_schedule(sched,net_actual[i],settle["emergency"],f"Q3-{issues}@{date}"))
            if i >= core.TEST_START:
                rows.append({"date":date,"plan":plan,"adjust":fg,"charge":fc,"discharge":fd,"soc":fs,"emergency":settle["emergency"],
                    "total_cost":settle["total_cost"]})
                metrics.append([date,"+".join(map(str,issues)) or "none",settle["total_cost"],settle["plan_cost"],settle["up_cost"],
                    settle["down_cost"],settle["emergency_cost"],settle["emergency_kwh"],settle["waste_kwh"],fs[0],fs[-1]])
            cols=["date","combo","total_cost","plan_cost","up_cost","down_cost","emergency_cost","emergency_kwh","waste_kwh","start_soc",
                "end_soc"]
            return rows,pd.DataFrame(metrics,columns=cols),checks,iterations

def metrics_forecast(actual, pred):
    mask=np.isfinite(pred)
    err=np.abs(actual[mask]-pred[mask])
    return {"MAE":float(err.mean()),"RMSE":float(np.sqrt(np.mean(err**2))),"WAPE":float(err.sum()/np.abs(actual[mask]).sum())}

def q1_sensitivity(net, price):
    rows=[]
    for eta in [0.80,0.85,0.90,0.95,1.00]:
        oldc,oldd=core.ETA_C,core.ETA_D; core.ETA_C=core.ETA_D=eta
        s=core.optimize_schedule(net,price,core.SOC_SAFE,terminal_eq=core.SOC_SAFE)
        rows.append(["效率",eta,float(np.sum(price*s.g))])
        core.ETA_C,core.ETA_D=oldc,oldd
    return pd.DataFrame(rows,columns=["parameter","value","cost"])

def make_plots(q1, load1, pv1, price1, pred, q2m, q3all, q43all, sensitivity):
    x=np.arange(T)/6
    plotdir=OUT/"records"/"plot_data"; plotdir.mkdir(parents=True,exist_ok=True)
    pd.DataFrame({"hour":x,"load_kw":load1,"pv_kw":pv1,"price_yuan_per_kwh":price1}).to_csv(plotdir/"fig01_input_profiles.csv",
        index=False)
    fig,ax=plt.subplots(2,1,figsize=(7.2,5.4),sharex=True); ax[0].plot(x,load1,label="负荷"); ax[0].plot(x,pv1,label="光伏"); ax
    [0].set_ylabel("功率/kW"); ax[0].legend(); ax[0].grid(alpha=.2); ax[1].plot(x,price1,color="#d97706"); ax[1].set(
        xlabel="时刻/h",ylabel="电价/(元/kWh)"); ax[1].grid(alpha=.2); core.savefig("fig01_input_profiles.pdf")
    fig,ax=plt.subplots(2,1,figsize=(7.2,5.4),sharex=True); ax[0].plot(x,q1.g,label="购电"); ax[0].plot(x,q1.charge,label="充电"
        ); ax[0].plot(x,q1.discharge,label="放电"); ax[0].legend(ncol=3); ax[0].set_ylabel("电量/kWh"); ax[0].grid(alpha=.2);
        ax[1].plot(np.arange(T+1)/6,q1.soc,color="#059669"); ax[1].set(xlabel="时刻/h",ylabel="SOC/kWh"); ax[1].grid(alpha=.2)
        ; core.savefig("fig02_q1_dispatch.pdf")
    pd.DataFrame({"hour":x,"grid_kwh":q1.g,"charge_kwh":q1.charge,"discharge_kwh":q1.discharge}).to_csv(plotdir/"
        fig02_q1_dispatch.csv",index=False)
    c3=q3all.groupby("combo").total_cost.sum().sort_values(); c43=q43all.groupby("combo").total_cost.sum().sort_values(); fig,
    ax=plt.subplots(1,2,figsize=(9,3.8)); ax[0].bar(c3.index,c3.values/1e6,color="#3b82f6"); ax[0].set(ylabel="费用/百万元"
        ,xlabel="启用发布时刻"); ax[0].tick_params(axis='x',rotation=35); ax[1].bar(c43.index,c43.values/1e6,color="#f59e0b");
        ax[1].set(xlabel="波动价下启用发布时刻"); ax[1].tick_params(axis='x',rotation=35); core.savefig("fig03_combo_comparison.
        pdf")
    pd.DataFrame({"combo":c3.index,"fixed_price_cost":c3.values,"variable_price_cost":[c43.get(k,np.nan) for k in c3.index]}).
        to_csv(plotdir/"fig03_combo_comparison.csv",index=False)
    i=79; fig,ax=plt.subplots(2,1,figsize=(7.2,5.2),sharex=True); ax[0].plot(x,core_load[i],label="实际"); ax[0].plot(x,pred['
        load'][i],label="预测"); ax[0].legend(); ax[0].set_ylabel("负荷/kW"); ax[1].plot(x,core_pv[i],label="实际"); ax[1].plot
        (x,pred['pv'][i],label="预测"); ax[1].set(xlabel="时刻/h",ylabel="光伏/kW"); ax[1].legend(); core.savefig("
        fig04_prediction_example.pdf")
    pd.DataFrame({"hour":x,"load_actual":core_load[i],"load_forecast":pred['load'][i],"pv_actual":core_pv[i],"pv_forecast":pred
        ['pv'][i]}).to_csv(plotdir/"fig04_prediction_example.csv",index=False)
    best=next(k for k in c3.index if k!="none"); z=q3all[q3all.combo=="none"].set_index("date").total_cost-q3all[q3all.combo==
        best].set_index("date").total_cost; fig,ax=plt.subplots(figsize=(7.2,3.8)); ax.hist(z,bins=35,color="#3b82f6",alpha
        =.85); ax.axvline(0,color="black",ls="--"); ax.set(xlabel=f"不调整费用 - 最佳非空组合({best})费用/元",ylabel="天数"); core.
        savefig("fig05_q3_daily_savings.pdf")
    z.rename("daily_saving").to_csv(plotdir/"fig05_q3_daily_savings.csv")
    sample=np.arange(core.TEST_START,len(core_dates),7); fig,ax=plt.subplots(figsize=(7.2,3.8)); ax.plot(core_dates.iloc[sample
        ],core_price4[sample].mean(1),label="实际日均电价"); ax.plot(core_dates.iloc[sample],pred['price'][sample].mean(1),
        label="历史滚动预测"); ax.set(ylabel="元/kWh"); ax.legend(); ax.grid(alpha=.2); fig.autofmt_xdate(); core.savefig("
        fig06_price_forecast.pdf")

```

```

pd.DataFrame({"date":core_dates.iloc[sample].astype(str).values,"actual_mean_price":core_price4[sample].mean(1),"
forecast_mean_price":pred['price'][sample].mean(1)}).to_csv(plotdir/"fig06_price_forecast.csv",index=False)
fig,ax=plt.subplots(figsize=(7.2,3.8)); ax.plot(pd.to_datetime(q2m.date),q2m.end_soc,color="#059669"); ax.axhline(core.
SOC_SAFE,color="black",ls="--",lw=.8); ax.set(xlabel="日期",ylabel="日末SOC/kWh"); ax.grid(alpha=.2); fig.
autofmt_xdate(); core.savefig("fig07_crossday_soc.pdf")
q2m[["date","start_soc","end_soc"]].to_csv(plotdir/"fig07_crossday_soc.csv",index=False)
fig,ax=plt.subplots(figsize=(6.6,3.8)); ax.plot(sensitivity.value,sensitivity.cost,marker="o"); ax.set(xlabel="充/放电效率",
ylabel="问题一费用/元"); ax.grid(alpha=.2); core.savefig("fig08_efficiency_sensitivity.pdf")
sensitivity.to_csv(plotdir/"fig08_efficiency_sensitivity.csv",index=False)

def main():
    global core_dates, core_load, core_pv, core_price4
    core.makedirs(); core.setup_plot(); np.random.seed(core.SEED)
    dates,labels,price1,load1,pv1,load,pv,forecast,price4=core.read_inputs()
    core_dates,core_load,core_pv,core_price4=dates,load,pv,price4
    checks_data,dq=core.validate_data(dates,price1,load1,pv1,load,pv,forecast,price4); core.save_cleaned(dates,load,pv,price4)
    pred={"load":prediction_matrices(load),"pv":prediction_matrices(pv),"price":prediction_matrices(price4)}
    pmetrics={k:metrics_forecast(v,pred[k]) for k,v in {"load":load,"pv":pv,"price":price4}.items()}
    (OUT/"records"/"prediction_metrics.json").write_text(json.dumps(pmetrics,ensure_ascii=False,indent=2),encoding="utf-8")

    net1=(load1-pv1)*DT; q1=core.optimize_schedule(net1,price1,core.SOC_SAFE,terminal_eq=core.SOC_SAFE)
    q1cost=float(np.sum(price1*q1.g)); q1base=float(np.sum(price1*np.maximum(net1,0)))
    wb=core.load_workbook(core.ATT/"附件5"/"result1.xlsx"); ws=wb["计划购电量"]
    for i,v in enumerate(q1.g,start=2): ws.cell(i,2,float(v))
    ws=wb["充放电量"]
    for j in range(6): ws.cell(j+2,2,float(q1.charge[j*24:(j+1)*24].sum())); ws.cell(j+2,3,float(q1.discharge[j*24:(j+1)*24].
sum()))
    ws.cell(2,5,float(q1.soc[0])); ws.cell(3,5,float(q1.soc[-1])); wb.save(OUT/"results"/"result1.xlsx")

    q2rows,q2m,q2checks,q2it=simulate_q2(load,pv,np.tile(price1,(len(load),1)),np.tile(price1,(len(load),1)),pred["load"],pred[
"pv"])
    q42rows,q42m,q42checks,q42it=simulate_q2(load,pv,price4,pred["price"],pred["load"],pred["pv"])
    combos=[tuple(c) for r in range(4) for c in itertools.combinations([6,12,18],r)]
    q3runs=[]; q43runs=[]; allchecks=q2checks+q42checks; allit=[]; rows3={}; rows43={}
    for combo in combos:
        r,m,ch,it=simulate_q3_combo(combo,load,pv,forecast,np.tile(price1,(len(load),1)),np.tile(price1,(len(load),1)),pred["
load"],pred["pv"],False); rows3[m.combo.iloc[0]]=r; q3runs.append(m); allchecks+=ch; allit+=it
        r,m,ch,it=simulate_q3_combo(combo,load,pv,forecast,price4,pred["price"],pred["load"],pred["pv"],True); rows43[m.combo.
iloc[0]]=r; q43runs.append(m); allchecks+=ch; allit+=it
    q3all=pd.concat(q3runs,ignore_index=True); q43all=pd.concat(q43runs,ignore_index=True)
    best3=q3all.groupby("combo").total_cost.sum().idxmin(); best43=q43all.groupby("combo").total_cost.sum().idxmin()
    core.fill_output("result2.xlsx","result2.xlsx",dates.iloc[core.TEST_START:],q2rows,False)
    core.fill_output("result3.xlsx","result3.xlsx",dates.iloc[core.TEST_START:],rows3[best3],True)
    core.fill_output("result4-2.xlsx","result4-2.xlsx",dates.iloc[core.TEST_START:],q42rows,False)
    core.fill_output("result4-3.xlsx","result4-3.xlsx",dates.iloc[core.TEST_START:],rows43[best43],True)

    q2m.to_csv(OUT/"records"/"q2_daily.csv",index=False); q42m.to_csv(OUT/"records"/"q42_daily.csv",index=False); q3all.to_csv(
OUT/"records"/"q3_combos.csv",index=False); q43all.to_csv(OUT/"records"/"q43_combos.csv",index=False)
    pd.DataFrame(allchecks).to_csv(OUT/"records"/"constraint_checks.csv",index=False); pd.DataFrame(allit,columns=["date","
combo","issue","iterations","runtime_s"]).to_csv(OUT/"records"/"iteration_log.csv",index=False)
    sensitivity=q1_sensitivity(net1,price1); sensitivity.to_csv(OUT/"records"/"q1_efficiency_sensitivity.csv",index=False)
    params={"seed":core.SEED,"dt_hours":DT,"eta_c":core.ETA_C,"eta_d":core.ETA_D,"soc_bounds":[core.SOC_MIN,core.SOC_MAX],"
initial_soc":"2025-01-01 00:00 = 6000 kWh","official_output_start":"2025-02-01","forecast":"strict expanding/rolling
history","margin_quantile":0.8,"q3_combos":[list(x) for x in combos],"selected_q3_combo":best3,"selected_q43_combo":
best43,"settlement":"plan + positive 0.5 down deviation + 1.5 up deviation + 5 emergency"}
    (OUT/"records"/"model_parameters.json").write_text(json.dumps(params,ensure_ascii=False,indent=2),encoding="utf-8")
    make_plots(q1,load1,pv1,price1,pred,q2m,q3all,q43all,sensitivity)

    c3=q3all.groupby("combo").total_cost.sum().sort_values(); c43=q43all.groupby("combo").total_cost.sum().sort_values()
    passrate=float(pd.DataFrame(allchecks).passed.mean())
    report=f"融合重算版计算结果报告

## 运行环境

- Python {platform.python_version()}, NumPy {np.__version__}, pandas {pd.__version__}, SciPy {core.scipy.__version__}.
- 固定随机种子 {core.SEED}; HiGHS 线性规划; 10 分钟时步。

## 数据与信息集

365 日、每日 144 点数据门禁全部通过。所有正式策略只读取决策时刻以前的数据; 2025 年 1 月用于预测与 SOC 状态热身, 正式输出为 2 月 1 日至 12 月 31
日共 334 日。

## 问题一

费用 **{q1cost:,.2f} 元**, 无储能基线 **{q1base:,.2f} 元**, 节省 **{core.pct(q1base,q1cost):.2f}%**, 购电量 **{q1.g.sum():,.2f} kWh

```

```

**。

## 问题二

负荷预测 MAE {pmetrics['load']['MAE']:.2f} kW**、WAPE {pmetrics['load']['WAPE']:.2%}**；光伏预测 MAE {pmetrics['pv']['MAE']:.2f} kW**、WAPE {pmetrics['pv']['WAPE']:.2%}**。融合策略费用 {q2m.total_cost.sum():.2f} 元**，紧急购电 {q2m.emergency_kwh.sum():.2f} kWh**。A 版完美信息结果只作不可实现下界，不进入正式工作簿。

## 问题三

对 6/12/18 三个更新时刻的 8 种组合逐一回溯。最低费用组合为 {best3}**，费用 {c3.iloc[0]:.2f} 元**；完全不调整费用 {c3['none']:.2f} 元**。正式 result3 使用最低费用组合。各组合完整记录见 q3_combos.csv。

## 问题四

价格预测 MAE {pmetrics['price']['MAE']:.4f} 元/kWh**、WAPE {pmetrics['price']['WAPE']:.2%}**。问题 4-2 费用 {q42m.total_cost.sum():.2f} 元**；问题 4-3 最低费用组合为 {best43}**，费用 {c43.iloc[0]:.2f} 元**，不调整费用 {c43['none']:.2f} 元**。

## 灵敏度与验证

问题一效率 0.80--1.00 的费用响应已记录。所有 {len(allchecks):}** 项逐日/逐组合约束回代通过率为 {passrate:.2%}**。费用分量、跨日 SOC、供需和储能边界均保存为机器可读记录。

## 图表

fig01 输入曲线；fig02 问题一调度；fig03 更新组合费用；fig04 预测示例；fig05 逐日调整收益；fig06 价格预测；fig07 跨日 SOC；fig08 效率敏感性。所有图均为矢量 PDF。

## 从零复现

```bash
MPLCONFIGDIR=mplconfig python3 code/run_fusion.py
```

REPORT.write_text(report,encoding="utf-8")
print(json.dumps({"status":"ok","q1_cost":q1cost,"q2_cost":q2m.total_cost.sum(),"q3_best":best3,"q3_cost":float(c3.iloc[0]),"q42_cost":q42m.total_cost.sum(),"q43_best":best43,"q43_cost":float(c43.iloc[0]),"checks":len(allchecks),"pass_rate":passrate},ensure_ascii=False))

if __name__ == "__main__":
    main()

```